DEEP SCAN M7

Контракты, сбои и пределы принуждения в публичном M7 MCP

Главный результат. Сервер действительно организует работу по фазам. Но запись о прохождении процесса не равна доказательству выполненного исследования. В проверенном коде повторные запросы, конкуренция, восстановление и копии памяти могут превращать незавершённую или отменённую работу в положительный статус.

О чём этот документ

Проверяются конкретная реализация и её публичные контракты: допуск переходов, сохранение состояния, статус знания и поведение после ошибок. Для каждого случая указаны вход, наблюдение, причина и граница вывода.

Объект: **DukeDeSouth/m7-mcp-oss**
Package: **3.18.0** • Публичный snapshot: **72a7640**

```
72a7640700dbd7cebc0d57eabe8253989af084ba
```

Ключевые входы, наблюдения и ссылки на исходники включены в текст. Приложения для чтения не требуются. Приведённые трассы сокращены и не являются готовыми тестовыми скриптами.

Подготовлено с применением Codex: анализ исходников, локальные проверки и отдельное рецензирование. Живые модели M7 не вызывались.

02 / ИСПОЛНИТЕЛЬНОЕ РЕЗЮМЕ

Вердикт в четырёх границах

Публичное обещание	Что установлено
Принудительные gates	Проверка текущего ID работает. Содержательная рефлексия не обязательна: ошибочный JSON превращается в continue.
Надёжный прогресс	Повтор и одновременная подача могут менять чужую фазу, дублировать завершение или испортить сохранённый цикл.
Память о степени знания	Invalidation, нулевая confidence, удаление и противоречия не сохраняют одинаковый смысл во всех путях чтения.
Продолжение в новом чате	Дисковое состояние существует, но ошибки записи и смена проекта могут выдавать успех без восстанавливаемого цикла.

Проверяемая формулировка документации: **«модель не может пропустить фазу, просто решив так»** (WHY_OPEN_SOURCE.ru.md:63-68). Ниже различаются порядок фаз, содержание исследования и контроль действий клиента.

Что это означает на практике

M7 можно рассматривать как процессный MCP-инструмент для кооперативного агента. В этом режиме шаблоны, порядок и напоминания могут быть полезны. Более сильное утверждение - что процесс нельзя содержательно обойти и его записи надёжно подтверждают исследование - проверенный snapshot не выдерживает.

Что не опровергнуто

Последовательность фаз реализована; правильный Gate-ID требуется в reflection-ветке. Сборка и штатные тесты работают. Найденные дефекты не доказывают отсутствие пользы инструмента; неизвестная приватная версия не проверялась.

> Ключевой разрыв: **состояние процесса → свидетельство исследования → право выполнить эффект**. Эти три вещи требуют разных механизмов. В M7 они местами сближаются сильнее, чем позволяет проверка.

Код: README.md:1-35 ↗

Код: docs/WHY_OPEN_SOURCE.ru.md:63-68 ↗

03 / ЧТО ИМЕННО БЫЛО ПРОВЕРЕНО

Метод, версия и предел охвата

Уровень	Основание
Зафиксированная версия	HEAD повторно проверен 28.10.2026: 72a7640; source clone соответствует этому commit.
Исполнение	Node 24.14.0. Сборка tsc успешна. Штатный runner: 11 верхних test-файлов прошли. Это не число assertions.
Протокол	SDK client/server через stdio; tools/list показал 38 инструментов. Часть контрпримеров выполнена настоящими tools/call.
Безопасность проверки	Linux bwrap: отдельная сеть, скрытый HOME, очищенная среда, синтетические ключи и disposable workspace.
Зависимости	npm ci --ignore-scripts. Нативные ветки, требующие lifecycle-сборки, этим не квалифицированы.
Покрытие	MCP dispatch, cycle/session, reflection/control, memory/persistence, cache/context, task graph/transfer и выбранные служебные инструменты.

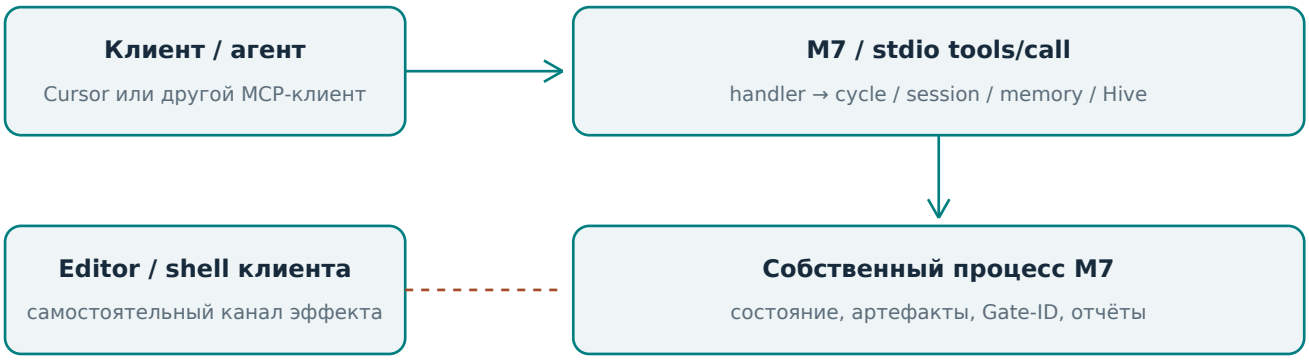
Как читать отметки

MCP - реальный протокольный вход. **Handler** - зарегистрированный обработчик, вызванный локально. **Core** - реальный класс/функция. **Статический** - вывод из исходников без утверждения о выполнении соответствующего полного маршрута.

Проверены также rewind/redo, граф задач, перенос и штатное закрытие MCP. Часть проверок состояния повторена в изолированной среде с исходниками только для чтения. PASS скрипта означает воспроизведение указанного наблюдения, а не правильность продукта.

Не проводились построчное ревью всех модулей, живые Hive-запросы, атакующие проверки удалённых серверов, тесты реального отключения питания и сравнение качества моделей. Число файлов не используется как мера качества.

Как устроен контроль M7



Пунктир: обязательная связь допуска эффекта в проверенном дереве не найдена.

Три самостоятельных уровня

1. Последовательность. Cycle хранит текущую фазу, артефакты и ожидание reflection. Внутри его API есть реальный автомат переходов.

2. Содержательность. Проверка длины и текущего Gate-ID связывает форму сообщения с фазой. Она не подтверждает, что приведённый вывод следует из прочитанного кода или что тест действительно выполнялся.

3. Исполнительный допуск. Чтобы гарантировать запрет правки до исследования, нужен контроль того канала, который меняет файл. MCP-сервер сам по себе не перехватывает независимые инструменты клиента.

Что найдено в исходниках

Класс M7CodeInterceptor описывает строгий допуск модификаций. В проверенном дереве найдены определения класса, но не его создание и вызов перед изменениями через редактор или shell. Обёртка registerTool вызывает handler напрямую; обязательная связь с редактором не показана.

Граница вывода: это отсутствие показанной интеграции в публичном дереве. Cursor end-to-end не запускался; неизвестная клиентская или приватная интеграция не объявляется невозможной.

Код: src/core/enforcement/codeInterceptor.ts:31-67 ↗

Код: src/index.ts:769-772 ↗

F01 / Пропущен смысл, а не порядок

Контракт, который проверяли

Reflection gate должен удерживать переход до осмысленного решения по результатам фазы. Контрпример разделяет проверку правильного ID и проверку содержания.

```
для каждой фазы:
  artifact_content = "x" * 101
  reflection = "## Gate-ID: <выданный ID>"

наблюдение: completedPhases.length = 7
reflection.decision = "continue"
reflection.confidence = 1.0
```

Почему это проходит

ID извлекается из Markdown и совпадает с ожидаемым. Затем JSON parser отвергает текст. Вместо отказа parseReflection возвращает default: continue, changeDetected=false, confidence=1.0. Фаза проходит по синтетической рефлексии, которой агент не предоставлял.

Что именно опровергнуто

Обязательность содержательной проверки. **Порядок семи фаз этим тестом не пропущен**: он формально пройден. Неправильный и отсутствующий ID контрольные вызовы правильно отвергают. Поэтому называть результат «nonce не работает» было бы ошибкой.

Штатный тест прямо ожидает auto-continue для неправильного JSON с правильным ID: поведение является закреплённой политикой, а не только случайной регрессией.

Условие исправления

Невалидная структура возвращает ошибку без перехода. Принятая reflection имеет явную схему и проверяемые ссылки на материал фазы. Само соблюдение схемы по-прежнему не доказывает истинность её содержания.

Код: src/core/orchestrator/cycleOrchestrator.ts:300-314 ↗

Код: src/core/orchestrator/cycleOrchestrator.ts:633-671 ↗

Код: tests/cycleOrchestrator.test.mjs:389-402 ↗

F02 / Запрос меняет собственный смысл

Минимальная последовательность

Завершить артефакт intake. Подать корректную reflection длиной более 100 символов. После успешного перехода в discovery повторить **тот же самый tools/call** с теми же аргументами.

```
1-й вызов: reflection intake → discovery
2-й вызов: тот же текст → artifact discovery

completedPhases = ["intake", "discovery"]
artifacts.discovery === previousReflection
```

Причина

У cycleTool один параметр artifact_content для двух операций. Handler выбирает значение сообщения из текущего состояния: если gate уже не ожидается, прежняя reflection становится новым артефактом. ID старого gate в этом пути не проверяется.

Почему это не только сценарий злонамеренного клиента

Клиент может повторить запрос после timeout, потери ответа или неясного завершения. Значение запроса при повторной доставке должно оставаться прежним. Здесь изменение серверного состояния меняет тип уже отправленного сообщения.

> Нарушен инвариант повторов: **повтор принятого запроса не должен создавать новую работу или закрывать другую фазу**. В данном случае discovery получает чужое содержимое.

Условие исправления

Развести submit_artifact и submit_reflection; требовать expected_phase, ожидаемую revision состояния и request_id. Результат уже принятого request_id возвращается повторно без новой мутации.

Код: src/core/tools/cycleTool.ts:87-102 ↗

Код: src/core/orchestrator/cycleOrchestrator.ts:163-183 ↗

F03 / Завершение не является терминальным

Сценарий A: повтор последнего артефакта

После нормального окончания implementation повторно подать финальный артефакт. Код вновь добавляет implementation в completedPhases и записывает состояние.

```
до повтора: completed = 7
после повтора: completed = 8
ответ / status: "Invalid count value: -2"
после reload: та же ошибка
```

Генератор progress вызывает repeat с отрицательным аргументом. Но сначала уже сохранён неверный список. Следовательно, это не просто косметическая ошибка форматирования: новое чтение воспроизводит повреждение.

Сценарий B: abort выглядит как успех

На первом gate подать правильный ID и decision=abort. Ответ сообщает полное завершение 7/7, хотя в состоянии есть лишь intake. После этого следующий артефакт того же цикла снова принимается.

```
abort response: "All 7 phases done."
actuallyCompletedPhases = ["intake"]
subsequentArtifactAccepted = true
```

Общий дефект модели

Один completed-флаг смешивает успех и прекращение. В CycleState нет устойчивого различения succeeded / aborted / failed, а admission не запрещает новые мутации после терминального перехода.

Условие исправления

Явный terminal outcome, уникальность завершённых фаз и отказ от повторной мутации терминального цикла. Formatter должен выводить реальное сохранённое состояние, а не константу 7/7.

Код: src/core/orchestrator/cycleOrchestrator.ts:181-241 ↗

Код: src/core/orchestrator/cycleOrchestrator.ts:363-374 ↗

Код: src/core/tools/cycleTool.ts:210-219 ↗

F04 / Два допуска одной фазы

Предусловие

Один активный цикл. Две допустимые подачи артефакта intake отправлены одновременно. Исходники и state-файлы не редактируются вручную.

```
Promise.all([submitArtifact(A), submitArtifact(B)])

bothResponsesSucceeded = true
completedPhases = ["intake", "intake"]
```

Почему однопоточный Node не спасает

Обработчики пересекаются в точках await. Оба могут прочитать одну текущую фазу и изменить общий объект до того, как конкурирующий переход станет видимым как занятый. Отдельная запись JSON не является транзакцией допуска.

Что доказано и что не расширяем

Выполнен конкретный дубль одной фазы через MCP. Это нарушает уникальность admission. Не измерялась вероятность на реальных IDE-клиентах; не утверждается, что каждый одновременный вызов обязательно портит данные.

> Полезный инвариант: при одном expected_state_revision допускается **ровно один** переход. Другой вызов получает conflict или прежний результат, но не второе завершение.

Проверка исправления

Сериализация по cycle ID или compare-and-swap состояния. Два параллельных вызова, повтор после неизвестного ответа и независимый читатель после перезапуска должны согласоваться по одному принятому переходу.

Код: src/core/orchestrator/cycleOrchestrator.ts:155-203 ↗

Код: src/core/orchestrator/cycleOrchestrator.ts:950-976 ↗

F05 / Session имеет второй путь к успеху

Это отдельный протокол

mcp_m7_session и семифазный mcp_m7_cycle зарегистрированы одновременно. Их требования и исходы нельзя считать одним автоматом.

Контрпример 1: ручное завершение

```
новая session: выполнено 0% требований
canCompleteSession → false
handler(action="complete_session")
→ "Session Completed Successfully"
```

Guard реально умеет отказать, но обработчик complete_session не использует этот отказ как обязательное условие завершения. Операция доступна модели как обычный tool input; отдельный допуск человека в рассмотренном маршруте не найден.

Контрпример 2: автоматическое продвижение

Для medium-session требования ранних фаз отмечены штатно. В implementation достаточно code_quality_verified для перехода к validation. Затем tests_passing завершает session, хотя performance, Playwright, success metrics, документация, prototype и integration остаются false.

Причина: getPhaseRequirements возвращает пустой набор для implementation и validation. Проверка every([]) истинна. Отсутствие требований интерпретируется как удовлетворение требований.

Условие исправления

Один обязательный completion-predicate для всех путей. Если override нужен, это отдельный outcome с явной причиной и полномочием, а не обычный «успех». Требования каждой фазы должны быть полными либо явно неприменимыми.

Код: src/core/orchestrator/taskOrchestrator.ts:511-520 ↗

Код: src/core/orchestrator/taskOrchestrator.ts:555-609 ↗

Код: src/core/tools/unified/sessionTool.ts:252-263 ↗

F06 / Смена проекта расщепляет владельца

Штатная последовательность

Открыть цикл в проекте A. В том же MCP-процессе вызвать init_project для проекта B. После успешного переключения начать новый цикл и сохранить его артефакт. Затем поднять новый MCP-процесс сразу в B и запросить этот цикл.

```
состояние нового цикла → A/.m7/cycles
артефакт нового цикла → B/m7-cycles
состояние в B → отсутствует
новый процесс B → "Cycle ... not found"
```

Причина

Singleton CycleOrchestrator связывает cyclesDir с projectPath при создании. Запись артефактов и reflection-логов использует меняющийся getProjectPath(). После переключения один task имеет разных владельцев хранения.

Дополнительное наблюдение

Reflection старого A-цикла после смены проекта записывается в лог под B. Это подтверждает смешение workspace-данных в поддерживаемом процессе. Разграничение пользователей или удалённая утечка этим не проверялись.

> Восстанавливаемость требует, чтобы **state, artifacts, log, cache и project identity** принадлежали одному зафиксированному владельцу. Одного глобального «текущего пути» для этого недостаточно.

Проверка исправления

Привязать cycle к immutable project identity. Смена workspace либо выбирает отдельный owner, либо полностью переинициализирует все зависимые singleton. A → B → restart B должен сохранять адресуемость и не смешивать журналы.

Код: src/core/orchestrator/cycleOrchestrator.ts:57-61 ↗

Код: src/core/orchestrator/cycleOrchestrator.ts:185-198 ↗

Код: src/core/utils/projectPath.ts:1 ↗

F07 / Разные циклы делят доказательства

Сценарий без подмены времени

Созданы два последовательных цикла с одинаковым описанием. У них разные taskId. Первый оставил INTAKE.md и DISCOVERY.md; второй записал новый INTAKE.md.

```
first.taskId != second.taskId
first.artifact.path == second.artifact.path

на диске:
  INTAKE.md    → второй цикл
  DISCOVERY.md → первый цикл
```

Почему это опаснее простого overwrite

Папка артефактов строится из taskSlug, а не уникального taskId. Старый JSON хранит первый intake, но ссылка на его Markdown теперь открывает второй. В одной папке появляется смесь фаз разных запусков, похожая на цельную историю.

Отдельная граница идентификатора

Сам taskId строится из первых 20 ASCII-символов slug и последних шести цифр миллисекундного времени. Два параллельных startCycle с разными окончаниями описания получили одинаковый ID при реальном Date.now, без подмены часов. Оба вернули инструкции. Повторная проверка подтвердила коллизию; сохранённый JSON содержал одну из задач вместо двух отдельных историй. Это Core-проверка; частота в IDE не измерялась.

Условие исправления

Неизменяемый уникальный ID в пути state и артефактов, revision/hash содержимого в ссылке, запрет тихого overwrite. Два одинаково названных задания должны оставлять две независимые истории.

Код: src/core/orchestrator/cycleOrchestrator.ts:931-947 ↗

Код: src/core/orchestrator/cycleOrchestrator.ts:185-198 ↗

F08 / Успех без сохранённого состояния

Ошибка записи превращается в нормальную инструкцию

В disposable fixture .m7/cycles создан как обычный файл. Запись состояния невозможна. startCycle всё равно возвращает штатную intake-инструкцию: saveState ловит исключение и только пишет warning.

```
startReturnedSuccessInstruction = true
freshInstance.success = false
freshInstance.error = "Cycle ... not found"
```

Почему это нарушает handoff

Положительный ответ сообщает клиенту, что цикл создан, но новый экземпляр оркестратора его не находит. Отдельный catch при записи артефакта тоже позволяет продолжить процесс без соответствующего файла. RAM-состояние и журнал на диске здесь не один commit.

Резервная копия не используется при parse failure

В другом тесте основной JSON существует и читается, но синтаксически повреждён. .bak.1 содержит валидные данные. safeRead возвращает читаемые байты основного файла; JSON.parse падает, а валидная копия не перебирается.

```
main = malformed JSON
backup = valid JSON
safeReadJSON → data:null, JSON parse error
```

Условие исправления

Не подтверждать переход, если необходимая запись не принята. Recovery должен валидировать кандидата целиком, различать not_found / corrupt / incompatible и перебирать резервные версии по определённому правилу. Физическое выключение питания этим тестом не моделировалось.

Код: src/core/orchestrator/cycleOrchestrator.ts:968-976 ↗

Код: src/core/utils/atomicWrite.ts:335-377 ↗

F09 / Недоверие не переживает reload

Точная последовательность

Создать и сохранить L2-запись с confidence=0.9. Вызвать invalidate_bulk. Прочитать текущий объект, затем заново загрузить storage и повторить чтение.

```
после invalidation:
  confidence = 0.27
  status = STALE
  invalidatedAt = <timestamp>

после reload:
  confidence = 0.9
  status = KNOWN
  invalidatedAt = null
```

Причина состоит из двух частей

Обработчик изменяет объект в cache без полного устойчивого сохранения. Кроме того, преобразование между unified- и legacy-записью не переносит весь invalidation-контекст. Добавления одного вызова save недостаточно для полноценного round-trip.

Семантический эффект

После новой сессии ранее отменённый вывод выглядит надёжнее, чем до перезапуска. Это дефект именно памяти о недоверии: сохранять текст знания недостаточно, если не сохраняется причина, по которой на него больше нельзя опираться.

Проверка исправления

store → invalidate → read → reload → read должен сохранять invalidate-событие, confidence и основание статуса. Нельзя выводить KNOWN из устаревшей копии, не учитывая более новое отменяющее событие.

Код: src/core/tools/unified/memoryTool.ts:826-835 ↗

Код: src/memory/auto/unified-memory-core.ts:411-430 ↗

Код: src/memory/auto/unified-memory-core.ts:1240-1264 ↗

F10 / Удалённое знание возвращается копией

Последовательность

Сохранить L2-знание, которому включено зеркалирование в CYCLE_REGISTRY.md. Удалить исходный record штатной операцией. Убедиться, что get(originalId) возвращает null. Затем выполнить поиск по уникальному маркеру содержимого.

```
DELETE → "Deleted: ... <originalId>"
get(originalId) → null
search(sentinel) → ref_CYCLE_REGISTRY.md_Knowledge
status = KNOWN; confidence = 0.9
```

Что именно нарушено

Удалён именно исходный ID; исторический Markdown остался. Поиск рассматривает этот исторический текст как новый источник с собственной высокой confidence. Он не знает, что тот же тезис уже был удалён из основного storage.

Это не утверждение о secure erase: такой контракт не установлен. Проблема состоит в **повторном использовании отменённого знания как актуального**. Пользовательская операция удаления и retrieval должны иметь согласованную семантику.

Условие исправления

Задать, что означает delete: скрытие ID, прекращение retrieval или физическое стирание. Для retrieval-удаления нужен tombstone/lineage до зеркал, либо исторический результат с явной недействительностью. Сохранённый архив допустим; его автоматическое повышение до KNOWN требует отдельного основания.

Код: src/memory/auto/unified-memory-core.ts:579-703 ↗

Код: src/memory/auto/unified-memory-core.ts:765-781 ↗

Код: src/core/tools/unified/memoryTool.ts:431-439 ↗

F11 / Противоречие зависит от тёплого cache

Одинаковая операция, разные предшествующие чтения

Две записи сохраняются с confidence=0.9. Публичный store_at получает dependency_specs с отношением contradicts к уже существующему ID. Сравниваются тёплый cache и reload перед тем же маршрутом.

```
warm: confidence = [0.54, 0.54]
cold: confidence = [0.90, 0.90]
cold statuses = [KNOWN, KNOWN]
```

Причина

Обработка contradicts берёт участника отношения непосредственно из cache. На холодном пути уже сохранённая запись туда не обязательно загружена. Логика статуса зависит от присутствия объекта в оптимизационном слое, а не только от хранимых фактов и отношений.

Инвариант

> **Cache может менять скорость, но не значение операции.** При одинаковой устойчивой базе и одинаковом запросе результат не должен зависеть от того, читали ли нужный ID раньше.

Тест не утверждает, что система умеет автоматически понимать смысл любого естественно-языкового противоречия. Здесь отношение contradicts явно передано самим API; нарушается уже обработка объявленной связи.

Проверка исправления

Разрешать endpoints отношений через единый persistent lookup, сохранять результат обновления обоих участников и проверять одинаковый исход в warm, cold и restart-сценариях.

Код: src/memory/auto/unified-memory-core.ts:367-397 ↗

Код: src/memory/auto/unified-memory-core.ts:743-752 ↗

F12 / Нулевая уверенность повышается сама

Новый контрпример

Через настоящий публичный handler store_at сохранить запись с confidence=0. Убедиться, что до reload значение равно нулю. Перезагрузить persistent memory и прочитать тот же ID.

```
stored: confidence = 0; status = UNKNOWN
reloaded: confidence = 0.5; status = ASSUMED
```

Класс ошибки

Нулевое числовое значение в пути конвертации воспринимается как отсутствие значения. Fallback через логическое OR подменяет допустимый 0 значением по умолчанию 0.5. Здесь отсутствует новый источник, новая оценка или решение человека, которое оправдывало бы повышение.

Почему это не косметика

Статус - часть публичного назначения памяти M7. Если после нового чата UNKNOWN превращается в ASSUMED, клиент получает другой уровень доверия к той же записи. Это независимый от invalidation дефект round-trip значений.

Условие исправления

Различать missing/null и число 0; применить nullish fallback и валидировать диапазон на записи и чтении. Проверить устойчивость 0, 0.01, пороговых значений и 1 при reload, backup recovery и legacy migration.

> Требование к памяти: **decode(encode(record))** сохраняет не только текст, но и confidence, invalidation, provenance и адресуемый ID.

Код: src/memory/auto/unified-memory-core.ts:1240-1264 ↗

F13 / Фильтр поиска обходит боковая копия

Новый контрпример

Сохранена L2-запись с confidence=0.1. Затем вызван публичный search с depth_levels=[L0] и confidence_threshold=0.99. Основной storage по этим условиям не должен возвращать данную L2-запись.

```
search:
  depth_levels = ["L0"]
  confidence_threshold = 0.99

result includes Markdown L2 reference:
  confidence = 0.9; status = KNOWN
```

Почему одного фильтра недостаточно

Поиск объединяет основной storage с извлечением знаний из Markdown. Сайд-канал назначает ссылочному результату свою confidence и не проводит тот же фильтр до выдачи. В результате общий ответ не удовлетворяет ограничениям самого запроса.

Что доказано

Выполнен конкретный публичный handler-вызов с неподходящим уровнем и числовым порогом. Это не измерение точности retrieval вообще; это нарушение проверяемого контракта фильтрации и источник повышения доверия при зеркалировании.

Условие исправления

Единый финальный predicate для каждого источника перед merge/rank. Источник, исходная confidence, уровень и историчность должны сохраняться. Если Markdown не имеет валидной оценки доверия, его нельзя молча считать KNOWN с 0.9.

Код: src/memory/auto/unified-memory-core.ts:579-703 ↗

F14 / Storage не сохраняет адрес записи

Публичный API принимает больше, чем надёжно хранит

Публичная схема store_at принимает L4; достижимость Core прослежена по source. В тесте две разные L4-записи созданы через реальный Core store. До reload они доступны через cache; после reload оба get по выданным ID возвращают null. Отдельный MCP-вызов этой fixture не выполнялся.

```
id1 != id2
reload()
get(id1) → null
get(id2) → null
project metadata содержит поля последней записи
```

Причина

L4 переводится в общий project metadata blob. В legacy store ключ записи не используется так же, как для record storage. После очистки cache unified get не восстанавливает ID-addressability из project blob.

Следовательно, нельзя писать «все байты исчезли»: часть полей остаётся. Точный дефект - успешный record API выдаёт адреса, которые устойчивое представление не умеет обслужить.

Условие исправления

Согласовать уровни публичного API с реальными формами storage. Если L4 - singleton metadata, не выдавать его за множество независимых record ID. Project identity должна входить в ключ и время жизни зависимостей.

Код: src/memory/auto/unified-memory-core.ts:1196-1203 ↗

Код: src/memory/memorySystem.ts:216-222 ↗

Код: src/memory/memorySystem.ts:293-296 ↗

F15 / «Точный» cache стирает направление

Два контрпримера exact hit

```
move users from archive to active
move users from active to archive
→ одинаковый ключ / первый ответ

Видалити резервні копії
Зберегти резервні копії
→ одинаковый ключ / первый ответ
```

Причина

Нормализация удаляет не-ASCII word-символы, отбрасывает короткие слова и сортирует оставшиеся токены. Порядок, направление и кириллица исчезают до хеширования. Это не криптографическая коллизия MD5: разные смыслы намеренно сведены к одной строке ещё до hash.

Контекст повторного использования

Planner возвращает cache success перед чтением текущего кода. Project/revision не входят в task key. Даже для похожего описания результат не обязан оставаться верным после смены проекта или изменений исходников.

Граница доказательства

Функция cache выполнена с синтетическим sentinel-ответом; интеграционный ранний return прослежен по source. Живой provider workflow и вредное исполнение возвращённого ответа не запускались.

Условие исправления

Exact cache сохраняет точные семантически значимые входы: текст, project/revision, роль и режим. Similarity - предложение для повторной проверки, а не готовый authoritative success. Нельзя лечить этот дефект только заменой алгоритма хеширования.

Код: src/hive/cache/taskResultCache.ts:76-86 ↗

Код: src/hive/cache/taskResultCache.ts:175-183 ↗

Код: src/hive/orchestrator/plannerOrchestrator.ts:407-427 ↗

F16 / Путь превращается в shell-код

Предусловие и безопасный тест

Выбран существующий POSIX workspace, в имени которого буквально содержится shell-подстановка. Код поиска релевантных файлов вставляет projectPath в строку child_process.exec. Изолированный тест создал только безвредный marker; сеть была выключена.

```
selected path содержит буквальные символы $()
findRelevantFiles(...)
→ shell подставляет команду из имени
→ markerCreated = true
```

Почему двойные кавычки не решают проблему

В shell подстановка команды выполняется и внутри двойных кавычек. Каталог здесь является данными, но интерпретатор получает его как часть программы. Фильтрации поисковых keywords недостаточно, когда отдельный аргумент остаётся shell-строкой.

Предел вывода

Нужен контроль выбранного пути или его метаданных. Доказан data-to-command переход в прямом вызове функции; путь от выбора workspace к context search прослежен статически. Не доказаны атака из интернета, повышение привилегий или выполнение произвольного текста обычной задачи.

Условие исправления

execFile/spawn с массивом аргументов без shell, ограниченные параметры и таймаут. Регрессионный тест должен включать пробелы, кавычки, доллары, backticks и перевод строки в допустимых POSIX-именах, сохраняя их данными.

Код: src/hive/context/fileSearcher.ts:31-55 ↗

F17 / Grounding не связывает claim с фактом

Два новых контрпримера

В памяти сохранена runtime-запись: развёртывание НЕ одобрено, confidence=0.9. Grounding получает обратный claim об одобрении со ссылкой на этот ID. Checker считает premise найденной и KNOWN; warnings отсутствуют.

```
evidence: "NOT approved"
claim: "approved"; ref: <existing ID>
→ found=true; status=KNOWN; warnings=0
```

Контрольный missing ref с явной связью к необратимому шагу вызывает strong warning. Если убрать premise_ids у шага, NOT_FOUND ещё виден в premise-отчёте, но warnings становятся нулевыми и итоговый текст сообщает достаточность оснований.

Причина и точный предел

Resolver проверяет существование и статус записи, а не отношение между её содержимым и claim. Проверка шагов обходит только перечисленные premise_ids; пустой список не создаёт требования покрытия. Это слабость обоснованности положительного отчёта.

Grounding в этом коде явно advisory: сам по себе он не обещает запретить развёртывание. Поэтому результат не следует называть обходом deployment security. Никакого deployment не выполнялось. Новая fixture использует реальные parser/resolver/checker/report API; интеграция cycle прослежена по коду.

Условие исправления

Развести reference exists, evidence status, claim support и step coverage. Непроверенная семантическая связь означает unverified, а отсутствие оснований у необратимого шага - incomplete. Положительный итог не должен скрывать эти границы.

Код: src/core/orchestrator/groundingGate.ts:98-127 ↗

Код: src/core/orchestrator/groundingGate.ts:212-251 ↗

Код: src/core/orchestrator/groundingGate.ts:313-315 ↗

F18 / Диагностика отдаёт часть ключей

Новый протокольный тест

mcp_m7_test_tool зарегистрирован в обычном tools/list. При tools/call он показывает часть OPENAI_API_KEY и ANTHROPIC_API_KEY из среды сервера. В тесте переданы только специально выдуманные значения; настоящие credentials не читались.

```
tools/list → mcp_m7_test_tool
фрагменты текстового tools/call ответа:
OPENAI_API_KEY: ... Present (FAKE_OPENAI_KEY...)
ANTHROPIC_API_KEY: ... Present (FAKE_ANTHROPIC_...)

статусный значок опущен; префиксы точны
```

Что установлено

Зарегистрированный MCP-инструмент возвращает secret-derived bytes в результат, который видит клиент/модель и потенциально сохраняет в transcript. Для проверки наличия ключа достаточно boolean present, без префикса.

Предел риска

Это доступ подключённого stdio-клиента к диагностике процесса. Не показаны сетевой listener, удалённый доступ без авторизации, отправка данных наружу, восстановление полного ключа или использование реального credential. Полный тестовый ключ в ответе отсутствовал.

Даже если диагностика намеренная, перенос части секрета из environment в tool output расширяет число мест, где эти байты хранятся и обрабатываются. Утверждение об ошибке минимизации достаточно; называть это полным компромиссом ключей было бы преувеличением.

Условие исправления

Оставить present/absent и безопасный статус конфигурации. Диагностические операции с чувствительной информацией отключать в стандартной регистрации. Проверить snapshot всех tool outputs на синтетических уникальных секретах.

Код: src/test-tool.ts:17-33 ↗

Код: src/index.ts:418-422 ↗

F19 / Напоминание видит не все вызовы

Что заявлено и как проверялось

README говорит о повторном вводе правил после N tool calls в одной фазе. Реализация и комментарий ruleReinjection уже: учитываются вызовы cycle. В fixture порог уменьшен с обычных 18 до 3; логика счётчика не изменялась.

```
start cycle → phaseMcpCallCount = 1
5 вызовов других MCP tools → count = 1
напоминания нет
2 cycle-status вызова → напоминание, count = 0
```

Как это интерпретировать

Напоминания работают при cooperative polling cycle. Но несколько других инструментов во время одной фазы не приближают порог. Если агент долго действует через иные инструменты и не запрашивает cycle, механизм не отслеживает весь этот промежуток.

Это узкое расхождение широкого текста документации с областью счётчика; не доказательство, что reinjection отсутствует. Влияние на качество кода или следование правилам не измерялось.

Условие исправления

Назвать точный контракт: «после N cycle calls», либо вынести учёт релевантных вызовов в общую dispatch-обёртку. Даже общий MCP-счётчик не видит независимые editor/shell-операции клиента без отдельной интеграции.

> Названный механизм и его фактическая область наблюдения - разные вещи. Гарантия должна ограничиваться теми событиями, которые счётчик действительно получает.

Код: src/core/orchestrator/ruleReinjection.ts:1-3 ↗

Код: src/core/orchestrator/cycleOrchestrator.ts:833-846 ↗

Код: src/index.ts:762-772 ↗

F20 / Предупреждение не равно экспертизе

Артефакт короче минимума всё же проходит

Трижды подан один символ. Результат: отказ, отказ, успех с открытым gate. Код прямо объясняет это предотвращением бесконечного цикла. Политика повышает живучесть, но ослабляет обязательное условие при повторе.

```
artifact = "x"
attempt1 → rejected
attempt2 → rejected
attempt3 → accepted
```

Adversarial review является мягкой проверкой формы

Проверяются заголовки и ключевые слова; неполный review возвращает предупреждение и явно не блокирует процесс. Это не независимый reviewer, который проверил исходники или истинность контраргумента. Сам код честно обозначает мягкое поведение.

Эпистемические метрики описывают журнал

Счётчики grounded premises, frontier и pivot/rewind вычисляются из уже записанных структур. Их корректная арифметика не превращает самоотчёт в независимое доказательство качества исследования. Pivot может быть улучшением, бесполезной сменой или ошибкой; одно число этого не различает.

Телеметрия: важная положительная граница

В проверенной ветке flush без настроенного endpoint не отправляет batch. Код разделяет согласие на телеметрию и наличие адреса отправки. Полный анализ всех сетевых маршрутов не проводился.

Условие исправления

Прямо разделять advisory / required / independently verified. При исчерпании попыток возвращать остановку или явный override, а не незаметно снижать критерий. Метрики процесса сопровождать независимой оценкой результата.

Код: src/core/orchestrator/cycleOrchestrator.ts:163-177 ↗

Код: src/core/orchestrator/adversarialGate.ts:63-74 ↗

Код: src/telemetry/collector.ts:368-375 ↗

F21 / Сводка выбирает старую рефлексию

Последовательность внутри одного цикла

Intake A принят. Discovery требует rewind в intake. Повторный intake B принят через pivot с новым framing; повторный discovery B тоже принят. Затем прочитана публичная инструкция architecture.

```
state/artifacts: новая версия B
currentFraming: B
Previous Phases / INTAKE:
  CONTINUE | OLD_A_ROOT_CAUSE_NOW_REFUTED
Previous Phases / DISCOVERY:
  REWIND | OLD_DISCOVERY_REJECTED_REDO_INTAKE
```

Разрыв между журналом и активной версией

Новые артефакты и framing на месте. Но buildCarryForward выбирает первую reflection совпавшей фазы через find. В таблице, которая ориентирует следующую фазу, остаются старый опровергнутый вывод и старое требование rewind; ссылки рядом уже открывают новые B-файлы.

Хранить историю нормально. Ошибка - подавать старую запись как актуальное резюме переработанной фазы без обозначения, что она заменена. Это другая ситуация, чем F07: здесь один taskId и законный пересмотр, а не два цикла с общей папкой.

Контроль и условие исправления

Первый rewind действительно вернул в intake, completedPhases очистился, pivot обновил framing, B-файлы были записаны. Значит дефект не в отсутствии redo, а в выборе версии для carry. Нужны phase-revision и явная связь active reflection → active artifact; исторические версии должны оставаться отдельно помеченными.

Код: src/core/orchestrator/cycleOrchestrator.ts:309-314 ↗

Код: src/core/orchestrator/cycleOrchestrator.ts:457-481 ↗

Код: src/core/orchestrator/cycleOrchestrator.ts:782-794 ↗

F22 / Ошибки длины расходуют право на rewind

Сценарий без гонки и подмены состояния

Дважды подан короткий intake-артефакт; оба запроса отвергнуты. Затем принят исправленный артефакт и корректная reflection. На discovery впервые запрошен rewind в intake.

```
short rejects in intake = 2
previous rewind requests = 0
request: rewind_target = intake
phaseAttempts.intake: 2 → 3
actual next phase = ARCHITECTURE
```

Причина

Один phaseAttempts используется для повторов проверки длины и для лимита возвратов. Rewind увеличивает счётчик целевой фазы; при превышении лимита код вместо возврата вызывает transitionToNextPhase от текущей discovery. Ошибки формы, уже исправленные пользователем, меняют смысл первого содержательного запроса пересмотра.

Контроль и граница

В соседней независимой fixture тот же первый rewind без двух коротких отказов возвращает в intake. Это не прежний случай «третья короткая подача проходит»: новый эффект - загрязнение бюджета семантического возврата. Намеренная защита от зацикливания существует, но счётчик считает разные события вместе.

> Нарушено соответствие управления: **принятый запрос вернуться к предпосылкам приводит к продвижению вперёд**. Требуется явный отказ или эскалация, если rewind больше не допускается.

Условие исправления

Разделить validationAttempts и rewindAttempts. Проверять лимит только соответствующих событий. Повторить тест с 0, 1 и 2 исправленными отказами: первый допустимый rewind должен иметь одинаковый смысл.

Код: src/core/orchestrator/cycleOrchestrator.ts:163-177 ↗

Код: src/core/orchestrator/cycleOrchestrator.ts:453-469 ↗

F23 / Порядок ребер меняет найденный граф

Четыре узла, четыре связи

Узлы A, B, C, D. Ребра в порядке вставки: A-B, B-C, C-D, A-C. Запрос: центр A, глубина 2, завершённые узлы включены. Кратчайший путь A-C-D имеет длину 2; D должен входить в окружение заданной глубины.

```
edges = [A-B, B-C, C-D, A-C]
getSubgraph(center=A, depth=2)
expected tasks = A, B, C, D
observed tasks = A, B, C
handler: 3 задач, 4 связей

те же ребра, но A-C первым → A, B, C, D
```

Почему DFS даёт неправильную границу

C сначала достигается по длинному A-B-C с нулевым оставшимся бюджетом. visited помечает C окончательно. Когда обход позже видит прямое A-C, повторное расширение запрещено, хотя теперь остаётся глубина для D. Алгоритм смешивает «уже посещён» и «уже посещён лучшим путём».

Доказательство и предел

Результат подтверждён через реальный task_graph handler и новый manager после reload. Меняется только порядок тех же связей. Mermaid при этом включает C-D без отдельной записи D в tasks; это дополнительная несогласованность проекции. Все варианты графов этим тестом не проверены.

Условие исправления

BFS по кратчайшим расстояниям либо повторный обход при большем остатке глубины. Перестановки ребер должны сохранять набор узлов. Публичный список, количество и диаграмма должны описывать один результат.

Код: src/core/task-graph/taskGraphManager.ts:292-340 ↗

Код: src/core/tools/taskGraphTool.ts:305-338 ↗

F24 / Перенос переписывает завершённую задачу

Условие коллизии отличается от cycle ID

TransferManager строит ID из slug заголовка и последних четырёх цифр Date.now. В fixture два разных source-task перенесены с одним заголовком при временах t и t+10 000 мс. Между ними первая целевая задача завершена.

```
transfer(source1, title, t) → task_title_0123
complete(task_title_0123, outcome)
transfer(source2, title, t+10000)
→ task_title_0123

after reload: status=active; old outcome absent
```

Причина и последствие

createTask без проверки присваивает nodes[id] новый объект. Повторный ID стирает статус и outcome прежнего target; обе ссылки переносов теперь ведут к одной заменённой идентичности. Отдельный контроль t+1 мс даёт иной ID.

Предел доказательства

Проверены реальные manager API и устойчивый граф. Date.now задан fixture детерминированно двумя возрастающими значениями; десять секунд физически не ожидались. Частота реальных столкновений не измерена. Путь handler к manager прослежен, но полный MCP transfer-прогон не выполнялся.

Условие исправления

Уникальный ID и отказ от duplicate create вместо тихого update. Проверить одноимённые переносы, одинаковый millisecond, интервал 10 000 мс и одинаковый обрезанный prefix: две истории и прежний completed outcome должны сохраниться после reload.

Код: src/core/task-transfer/transferManager.ts:189-200 ↗

Код: src/core/task-graph/taskGraphManager.ts:125-151 ↗

Код: src/core/tools/taskTransferTool.ts:103-115 ↗

F25 / Неуспешный перенос оставляет задачу

Минимальная отрицательная операция

executeTransfer вызван с отсутствующим source ID. Ожидание: отказ до создания целевой задачи. Наблюдение: новый target уже сохранён, затем updateTaskStatus не находит source и выбрасывает ошибку; наружу возвращается success=false.

```
executeTransfer("missing-source", ...)
→ success=false
→ error="Task not found: missing-source"

tasks before = 9
tasks after  = 10
fresh manager: orphan target is active
```

Причина

Создание target, обновление source и связь выполняются последовательными мутациями с отдельными saves. Общего commit или rollback нет. Отказ операции не соответствует отсутствию её устойчивых эффектов.

Граница обычного использования

Это manager-level отрицательный тест. AnalyzeTask действительно умеет регистрировать задачи в графе; утверждение «граф всегда пуст» отвергнуто. Реальный handler выбирает session и передаёт её ID в manager; сквозной случай рассогласованной session/graph здесь не воспроизводился. Поэтому находка не означает, что каждый штатный перенос оставляет orphan.

Условие исправления

Проверить source и допустимость перехода до любых записей, затем принять одну полную новую версию графа. Невалидный source, повтор operation ID и ошибка записи должны оставлять либо прежний граф, либо один целиком принятый перенос.

Код: src/core/task-transfer/transferManager.ts:189-215 ↗

Код: src/core/task-transfer/transferManager.ts:247-252 ↗

Код: src/core/task-graph/taskGraphManager.ts:210-219 ↗

F26 / MCP подтвердил запись, закрытие её потеряло

Новая проверка обычного закрытия клиента

Через tools/call store_at принят синтетический L2-record с confidence=0.9. После положительного ответа вызван обычный SDK client.close(), который завершает дочерний процесс сигналом. Затем новый MCP-процесс в том же workspace выполняет get по выданному ID. Отдельный stdin-EOF сценарий не запускался.

```
store_at → success (ответ через 42 мс)
SDK client.close()
fresh MCP process → get(id) → not found

контроль: после success ждать 1200 мс
→ close → fresh process → record found
```

Причина

MemorySystem откладывает longterm/conceptual writes на debounce 1000 мс. Серверный gracefulExit непосредственно вызывает process.exit(0), без ожидаемого drain memory writes. Положительный tool response в этом окне означает принятие в RAM, а не завершённое устойчивое сохранение.

Отдельная библиотечная граница

При автоматическом flush после 50 записей даже MemorySystem.shutdown может вернуться до уже запущенного flush: в тесте primary-файл сразу содержал 1 запись, через 100 мс - 51. pendingWrites уже очищен, а отдельный in-flight Promise shutdown не ожидает. Это Core-наблюдение с зависимостью от расписания.

Условие исправления

Явно различать accepted и durable. При штатном закрытии ожидать pending и in-flight writes, затем завершать процесс. Контрольный тест должен получить подтверждённую запись после немедленного close/restart. Физическое отключение питания не моделировалось; потеря воспроизведена при обычном закрытии.

Код: src/index.ts:14-23 ↗

Код: src/index.ts:162-166 ↗

Код: src/memory/memorySystem.ts:49-103 ↗

Код: src/memory/memorySystem.ts:109-151 ↗

Хранение имеет предел, даже когда запись верна

Longterm ограничен не только в RAM

Через реальный MemorySystem API сохранён первый longterm-record и проверено его наличие в primary JSON. Затем добавлены ещё 2000 записей. Для отделения от flush-гонки каждые 49 операций выполнялся завершённый shutdown/drain.

```
first durable record: present
+ 2000 later records
primary longterm.json: 2000 entries
original ID: absent
fresh instance retrieve(original) → null
```

Причина: BoundedMap ёмкостью 2000 является одновременно рабочим storage, а persistLevel записывает всю текущую map. Вытеснение из памяти становится вытеснением из основного файла. Бэкапы не объявляются отсутствующими; secure erase не проверялся.

Старый незавершённый cycle тоже очищается

В отдельной fixture изменён только mtime синтетического cycle state на возраст восемь дней. Запуск нового цикла выполняет cleanupOldCycles и удаляет старый state независимо от его незавершённости. Продолжение старого ID возвращает not found, папка артефактов остаётся.

Как правильно классифицировать

> Это явные политики ограниченного хранения, **не доказательство скрытой атаки и не новые security findings**. Но «лежит на диске» не означает «сохраняется неограниченно»: срок, capacity, pin активной задачи и процедура архива должны входить в пользовательский контракт.

Возраст в тесте задан через filesystem mtime; восемь суток физически не ожидались. Для completed/inactive данных очистка может быть разумной. Проверяемый вопрос - что происходит с ещё нужной историей и как об этом узнаёт клиент.

Код: src/memory/memorySystem.ts:62-70 ↗

Код: src/memory/memorySystem.ts:435-441 ↗

Код: src/core/orchestrator/cycleOrchestrator.ts:64-100 ↗

Повторяющийся архитектурный разрыв

Потерянное различение	Наблюдаемое следствие
Форма / содержание	Gate-ID с невалидной reflection даёт continue; KNOWN записи не подтверждает противоположный claim.
Запрос / текущее ожидание	Повтор reflection становится следующим артефактом.
Завершён / успешен / прерван	Abort сообщает 7/7 и остаётся изменяемым.
Описание / идентичность	Одинаковые slug делят артефакты; временной suffix допускает taskId-коллизию.
Cache / устойчивый факт	Противоречие зависит от warm/cold; invalidation теряется после reload.
Историческая копия / актуальное знание	Удалённый или слабый тезис возвращается как KNOWN через Markdown.
Выполнено / записано / показано	Save failure скрыт; неверное состояние сохраняется до ошибки formatter.
Исторический / текущий вывод	После redo активная сводка остаётся привязана к старой reflection.
Принято / устойчиво сохранено	MCP success перед debounce не переживает штатное закрытие.

Архитектурный диагноз

Общий наблюдаемый паттерн - несколько неодинаковых представлений процесса: JSON state, Markdown, cache, session, cycle и готовый текст ответа. Это архитектурная интерпретация результатов, а не доказательство одной причины всех ошибок: OR-fallback, коллизия ключа и конкурентная запись имеют разные непосредственные причины. Каждый слой местами подтверждает локальный успех; согласованный внешний результат из этого не следует.

> Нужен не более длинный prompt, а чёткое правило: **какой факт является достаточным для какого перехода, кто его принимает и какая устойчивая запись подтверждает принятие**.

Требования, которые закрывают причины

Приоритет	Контракт и проверка
1. Честный исход	Success/abort/failure различены. Нет позитивного ответа при parse/save failure. Финальный повтор не мутирует state.
2. Один переход	Явные action+phase+revision+request_id. Один admission; rewind-лимит не расходуют старые ошибки длины.
3. Один владелец	Project/task ID неизменяемы. Active carry связан с текущей revision; transfer не заменяет старую identity.
4. Сохранение статуса знания	0, invalidatedAt, relation и tombstone переживают конвертацию и reload. С прогретым и пустым кэшем результат одинаков.
5. Единый поиск	Каждый источник соблюдает уровни и confidence_threshold. Историческая копия не повышает доверие автоматически.
6. Честные основания	Reference exists, supports claim и covers step - разные статусы. Непроверенное не выводится как достаточное.
7. Минимум полномочий	Нет shell-интерполяции данных и key-prefix в обычных tool outputs. Реальный эффект имеет показанный controller.
8. Полное сохранение	Отказ переноса не оставляет новой задачи. Close ожидает отложенные и уже начатые записи. Срок и объём хранения названы явно.

Как оценивать пользу после ремонта

На одинаковых задачах и начальном состоянии проверять принятые исправления, регрессии, время, стоимость и вмешательства человека. Отдельно оценить порядок фаз, память и Hive. Исправление протокольных дефектов ещё не доказывает выигрыш качества; количество успешных циклов не заменяет проверку результата.

Это требования к конкретным обещаниям M7, а не сравнительный рейтинг инструментов и не предложение построить другую платформу.

Сокращённые трассы проверок

Reflection fail-open через публичный MCP

```
tools/call mcp_m7_cycle:
  {"task_description":"audit fixture"}
для каждой из 7 фаз:
  {"task_id":id,"artifact_content":"x".repeat(101)}
  если ответ содержит Gate-ID:
    {"task_id":id,
     "artifact_content":"## Gate-ID: " + currentGate}
проверить: финальный ответ сообщает 7/7
```

Это сокращённая трасса протокола, не готовый SDK-клиент. currentGate нужно получать из каждой фактической инструкции; это не угадывание nonce. Проверка не утверждает, что фазы перескочены: она проверяет отсутствие содержательной reflection.

State-инварианты без живой модели

```
store_at(confidence=0) → reload → get(id)
ожидалось 0; получено 0.5

store_at(confidence=.9) → invalidate_bulk
→ reload → get(id)
ожидался STALE; получено KNOWN

search(depth_levels=[L0], threshold=.99)
получен ref Markdown: L2 / .9 / KNOWN
```

Правильная форма ответа на находку

Достаточны исправленный commit и повтор соответствующего теста; либо точный более узкий контракт, показывающий, что предпосылка находки неверна. Ссылка на приватную исправленную версию без доступного поведения не меняет вывод о зафиксированном публичном snapshot.

Для безопасного повтора используйте disposable workspace, фиктивные ключи, отдельную сеть и скрытый HOME. Не запускайте проверяемый сервер с реальной историей проектов или личными credentials.

Что выдержало проверку

Проверенные положительные свойства

• Публичный snapshot собирается; stdio MCP отвечает и регистрирует 38 инструментов.
• Штатный верхний test runner завершил 11 файлов без failures.
• Неверный и отсутствующий текущий Gate-ID отвергаются.
• Порядок rewind защищён проверкой направления; lead о произвольном прыжке вперёд отклонён.
• Reinjection срабатывает в своей узкой области cycle polling.
• У memory, relations, артефактов и процессов есть реальная реализация.
• Remote batch телеметрии требует настроенного endpoint в проверенной ветке.

Что осталось неизвестным

Живая эффективность Hive; точность всех TSI/Mesh/architecture веток; все возможные клиентские интеграции; частота дефектов при реальном использовании; текущая приватная версия. Полнота проверки всего дерева исходников не заявляется.

Итог

> **На проверенном commit M7 - реальный процессный сервер с существенными нарушениями корректности и достоверности собственных статусов.** Заявления об обязательном исследовании, сохранности статусов знания и надёжной передаче состояния требуют уточнения или исправления показанных маршрутов.

Вывод допускает проверяемое уточнение: другой результат на том же commit, ошибка в предпосылке либо исправленный код. Для каждого случая указаны границы, в которых он был проверен.

Навигация по первичным источникам

Все технические ссылки на предыдущих страницах закреплены на одном commit, а не на меняющейся ветке main. Номера строк - часть locator, не гарантия того, что будущая версия ведёт себя так же.

Описание и ограничения
README.md ↗

Публичный контракт
docs/WHY_OPEN_SOURCE.ru.md ↗

MCP entry / registerTool
src/index.ts ↗

Публичный cycle handler
src/core/tools/cycleTool.ts ↗

Фазовый автомат и сохранение
src/core/orchestrator/cycleOrchestrator.ts ↗

Session requirements
src/core/orchestrator/taskOrchestrator.ts ↗

Session handler
src/core/tools/unified/sessionTool.ts ↗

Unified memory
src/memory/auto/unified-memory-core.ts ↗

Публичный memory handler
src/core/tools/unified/memoryTool.ts ↗

Legacy memory storage
src/memory/memorySystem.ts ↗

JSON recovery
src/core/utils/atomicWrite.ts ↗

Grounding
src/core/orchestrator/groundingGate.ts ↗

Reinjection
src/core/orchestrator/ruleReinjection.ts ↗

Task result cache
src/hive/cache/taskResultCache.ts ↗

Planner early cache return
src/hive/orchestrator/plannerOrchestrator.ts ↗

File search
src/hive/context/fileSearcher.ts ↗

Диагностический tool
src/test-tool.ts ↗

Граф задач
src/core/task-graph/taskGraphManager.ts ↗

Публичный graph handler
src/core/tools/taskGraphTool.ts ↗

Менеджер переноса
src/core/task-transfer/transferManager.ts ↗

Проверенный commit 72a7640 ↗

Редакция 3 / 28.10.2026. Публичный снимок M7 MCP 3.18.0. Входы и результаты в документе сокращены; полный построчный и сетевой аудит не заявляются.